

Here's a **line-by-line explanation** of your OpenMP-based C++ program that calculates the **sum, max, min, and average** of an array **in parallel**:

Header Files

```
cpp
CopyEdit
#include <iostream>    // For input/output operations
#include <vector>      // For using std::vector
#include <omp.h>       // For OpenMP parallelization
```

Parallel Sum

```
cpp
CopyEdit
long parallelSum(const vector<int>& arr) {
    long sum = 0;
    #pragma omp parallel for reduction(+:sum)
    for (size_t i = 0; i < arr.size(); ++i) {
        sum += arr[i];
    }
    return sum;
}
```

- Uses **OpenMP's reduction(+:sum)** to safely accumulate `sum` across threads.
 - Automatically handles splitting the loop and merging results.
-

Parallel Max

```
cpp
CopyEdit
int parallelMax(const vector<int>& arr) {
    int max_val = arr[0];
    #pragma omp parallel for reduction(max:max_val)
    for (size_t i = 1; i < arr.size(); ++i) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }
    return max_val;
}
```

- Uses **reduction(max:max_val)** to find the maximum value.
 - Each thread tracks its local max, and OpenMP combines them.
-

Parallel Min

```

cpp
CopyEdit
int parallelMin(const vector<int>& arr) {
    int min_val = arr[0];
    #pragma omp parallel for reduction(min:min_val)
    for (size_t i = 1; i < arr.size(); ++i) {
        if (arr[i] < min_val) {
            min_val = arr[i];
        }
    }
    return min_val;
}

```

- Works like max but for the minimum.

Parallel Average

```

cpp
CopyEdit
double parallelAverage(const vector<int>& arr) {
    long sum = parallelSum(arr);
    return static_cast<double>(sum) / arr.size();
}

```

- Uses the previously defined `parallelSum`.
- Returns the **mean** as a double.

Main Function

```

cpp
CopyEdit
int main() {
    ...
}

```

Step-by-step:

1. Prompts user for number of elements.
2. Reads input into a `vector<int>`.
3. Calls the 4 parallel functions:
 - o `parallelSum`
 - o `parallelMax`
 - o `parallelMin`
 - o `parallelAverage`
4. Displays the results.

Summary of OpenMP Features Used:

Operation	OpenMP Directive Used	Description
Sum	<code>reduction(+:sum)</code>	Adds values in parallel
Max	<code>reduction(max:max_val)</code>	Finds max in parallel
Min	<code>reduction(min:min_val)</code>	Finds min in parallel

Make sure to **compile with OpenMP** using `-fopenmp` flag:

```
bash
CopyEdit
g++ -fopenmp your_program.cpp -o your_program
```